

Estudo Dirigido: O Microcontrolador STM32F411

Este estudo dirigido tem como objetivo consolidar os conceitos introduzidos no capítulo Os Microcontroladores STM32 e a Série F4. A transição da teoria arquitetural geral para o hardware real exige o domínio da navegação pelo **Reference Manual (RM0383)** e pelo **Datasheet** do microcontrolador.

As respostas devem ser redigidas em um arquivo `ESTUDO-DIRIGIDO-03.md`, salvo na raiz do seu repositório `seml-entregas`, seguindo o padrão já estabelecido, e submetidas via `git push origin master`.

i Orientações

- **Respostas autorais:** elabore explicações próprias com base nos manuais fornecidos.
- **Consulta Obrigatória:** Para as seções I e II, mantenha o RM0383 aberto. As páginas ou seções indicadas servem como ponto de partida.

I. Mapa de Memória e Barramentos

O processador Cortex-M4 não interage diretamente com pinos de entrada e saída; ele se comunica com controladores de periféricos (como o GPIO) através de uma matriz de barramentos internos da fabricante ST. A configuração de qualquer periférico começa por descobrir o seu **endereço base** no mapa de memória.

1. **Localização do GPIO:** No RM0383, localize a *Seção 2.3 (Memory map)*. Encontre a tabela *STM32F411xC/E register boundary addresses*. Qual é o endereço base (em hexadecimal) correspondente ao periférico **GPIOA**? E ao **GPIOC**?
2. **Topologia de Barramentos:** A mesma tabela informa em qual barramento cada periférico está pendurado. Em qual barramento o GPIOC está pendurado (ex: APB1, APB2, AHB1)?
3. **Frequência Máxima:** Como discutido na teoria, os barramentos operam em velocidades diferentes. Baseado na Figura 1 (*System architecture*) da Seção 2.1, ou nas especificações gerais do RM, qual é a frequência máxima permitida para o barramento **APB1**? E para o barramento **APB2**? Por qual motivo a ST divide os periféricos em barramentos de velocidades distintas em vez de conectar todos ao barramento mais rápido?

II. O Sistema de Clock (RCC)

Ao sofrer um *reset*, o STM32F411 não inicia utilizando um cristal externo nem operando na sua frequência máxima de 100 MHz; o sistema de *clock* é flexível e requer configuração explícita pelo software.

4. **Clocks Iniciais:** Consulte a Seção 6.2 (*Clocks*) do RM0383. Ao sair do reset, qual fonte de *clock* o sistema seleciona como **SYSCLK** (System Clock) por padrão? Trata-se do HSI (*High-Speed Internal*), do HSE (*High-Speed External*) ou do PLL? Qual é a frequência desse sinal padrão em MHz?

5. **Cristal da Blackpill:** Na nossa placa de desenvolvimento (a *Blackpill*), existe um cristal prateado soldado próximo ao microcontrolador. Ele atua como HSE. Consulte o esquema da placa ou pesquise: qual é a frequência exata (em MHz) desse cristal externo?

III. O Executável: Do Compilador ao Hardware

Nas práticas das Semanas 2 e 3, a cadeia de ferramentas de compilação gera e manipula diferentes formatos de arquivos antes de o código chegar de fato à memória do microcontrolador.

6. Na prática de inicialização, gravamos o programa na placa por meio do utilitário `st-flash`, que recebeu como entrada o arquivo `embedded.bin`. Posteriormente, verificamos o funcionamento do programa através do GDB, que recebeu o arquivo `embedded.elf`. Compare os dois arquivos (você pode utilizar o comando `ls -l` em ambos no terminal, além do `file`).
- O que o `.elf` contém internamente que o `.bin` não contém? (Relembre a tabela de seções e as partes de um arquivo objeto/executável Linux).
 - Por qual razão o gravador (`st-flash`) necessita fundamentalmente do formato `.bin` para gravar a imagem na memória Flash?
 - Por que o depurador (`gdb`) é incapaz de operar apenas com o `.bin`, necessitando da leitura do formato `.elf` original gerado pelo *linker*?